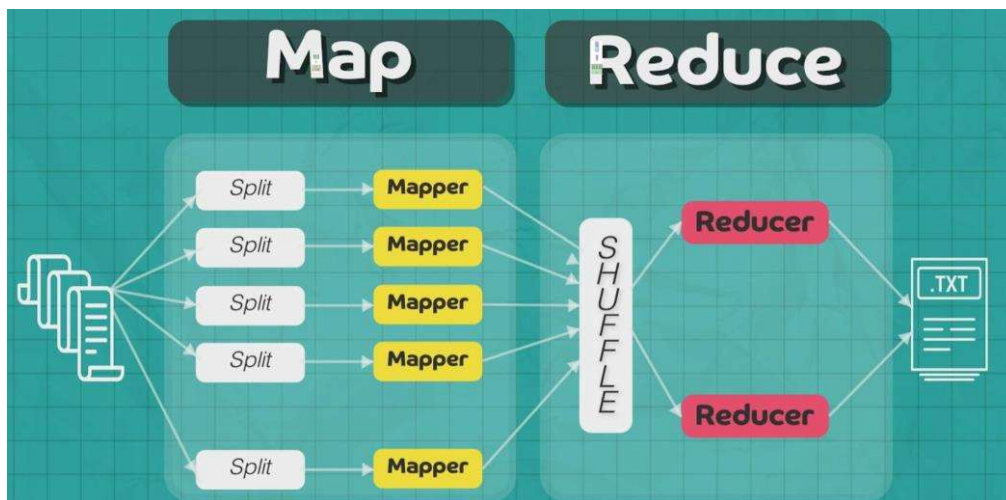


Module-3 Notes

Map-Reduce

MapReduce is a programming model and processing framework for distributed computing, invented by Google, that allows for processing large data sets with a distributed algorithm on a cluster. The MapReduce framework operates primarily in two phases: the Map phase and the Reduce phase.



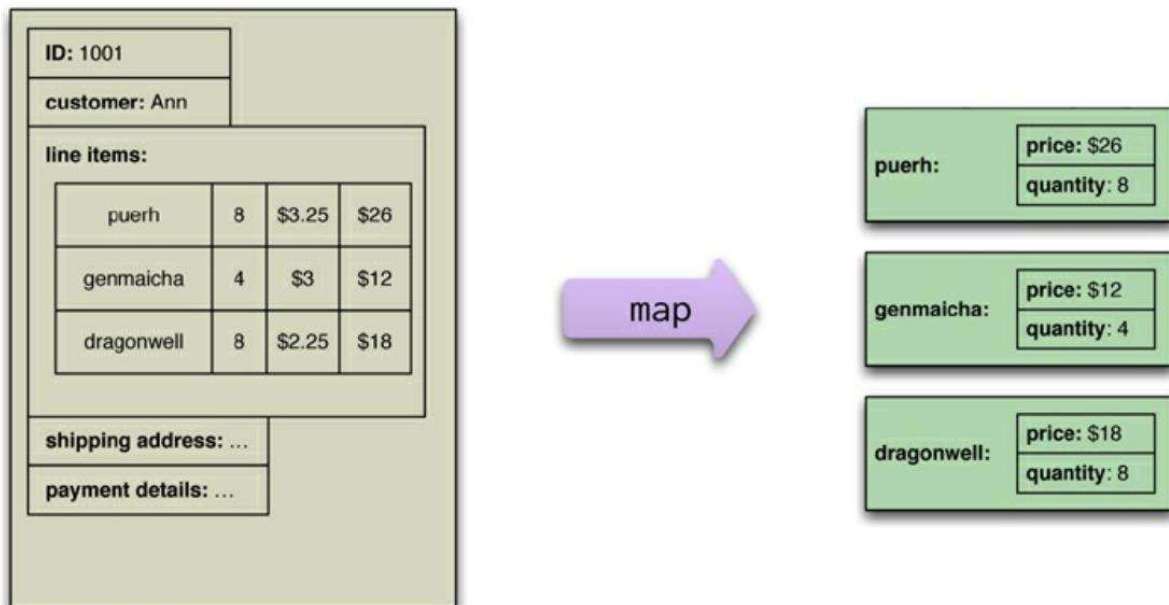
- ✚ Map Function: The Map function processes a key/value pair to generate a set of intermediate key/value pairs. It is applied to each input record in parallel.
- ✚ Reduce Function: The Reduce function merges all intermediate values associated with the same intermediate key. It processes each group of intermediate key/value pairs to generate the final output.

Workflow:

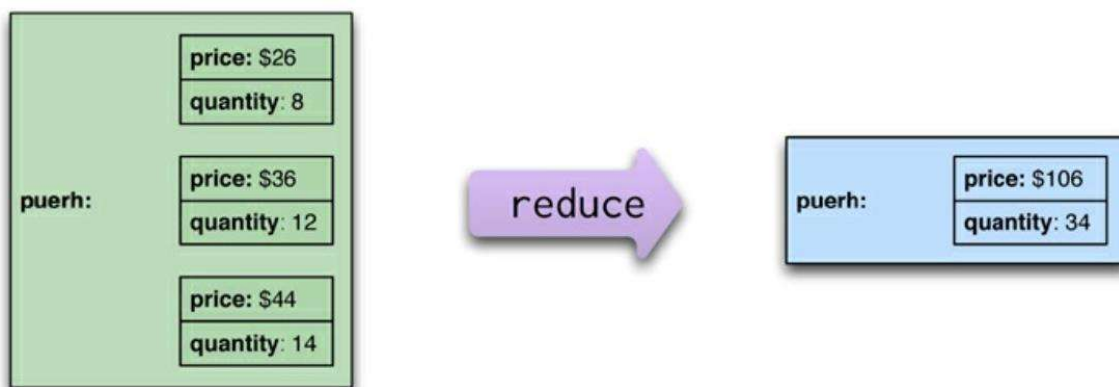
1. **Splitting:** The input data is split into manageable chunks, usually by a distributed file system like Hadoop Distributed File System (HDFS).
2. **Mapping:** The Map function is applied to each chunk, generating intermediate key/value pairs.
3. **Shuffling and Sorting:** The framework sorts and groups the intermediate data by key.
4. **Reducing:** The Reduce function is applied to each group of intermediate data to produce the final output.
5. **Output:** The final results are written to an output store, which can be HDFS or another distributed storage system.

Basic Map-Reduce

The first stage in a map-reduce job is the map. A map is a function whose input is a single aggregate and whose output is a bunch of keyvalue pairs. In this case, the input would be an order. The output would be key-value pairs corresponding to the line items. Each one would have the product ID as the key and an embedded map with the quantity and price as the values



A map function reads records from the database and emits key-value pairs

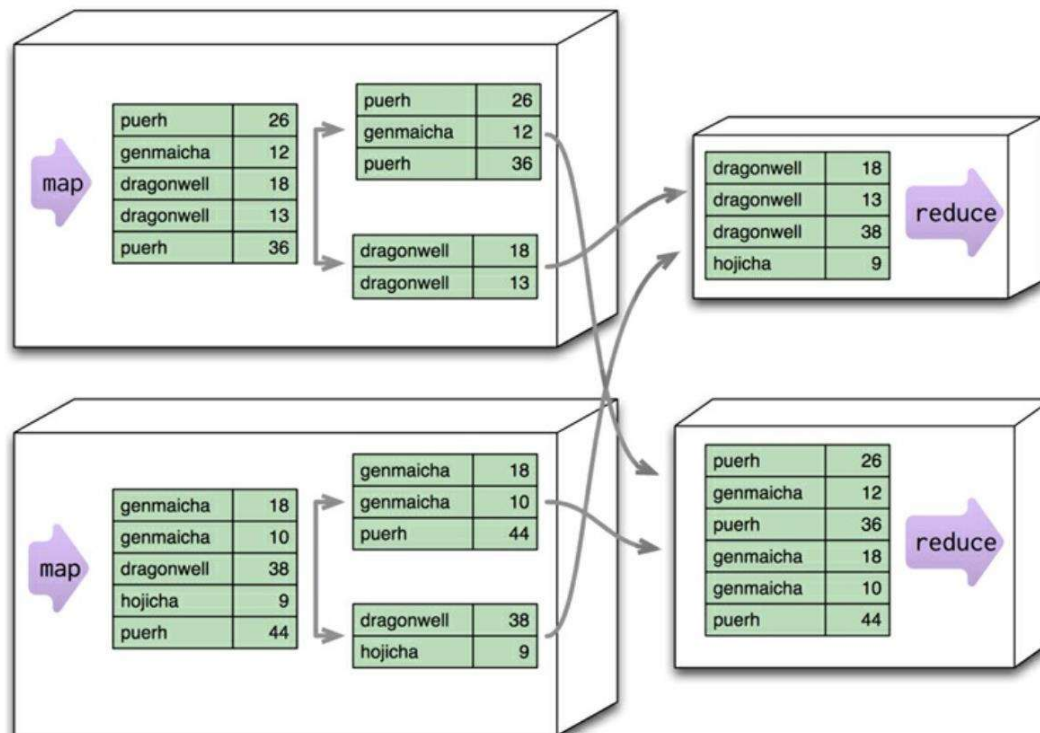


A reduce function takes several key-value pairs with the same key and aggregates them into one.

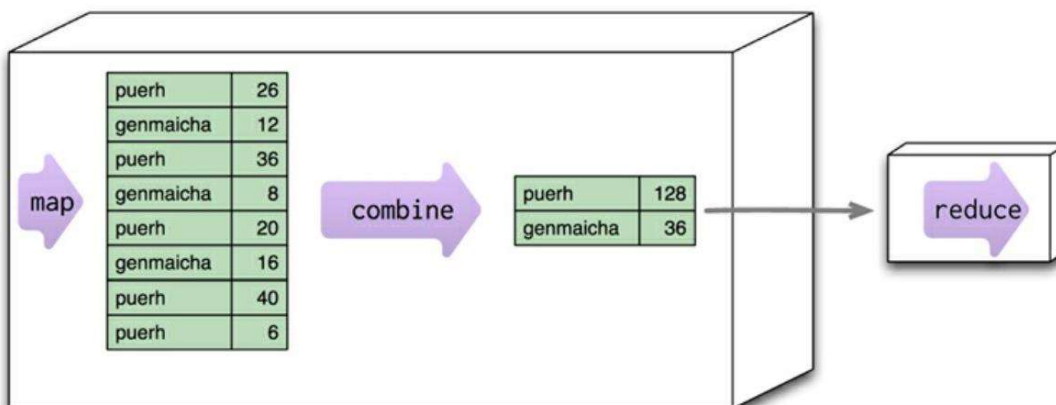
Partitioning and Combining

In the MapReduce framework, partitioning and combining play crucial roles in optimizing the processing of data. Partitioning involves distributing the intermediate key/value pairs generated by the map phase to reducers in an efficient manner. This is typically handled by a partitioner function, which determines the reducer that will handle each intermediate key. A common approach is to use a hash function on the key to ensure an even distribution across reducers, thus balancing the load and avoiding bottlenecks. Combining, on the other hand, is an optimization technique where a combiner function, which is essentially a mini-reducer, is applied to the intermediate data before it is sent to the reducers. This helps to reduce the amount of data shuffled across the network by combining the intermediate key/value pairs locally on each mapper. By performing local aggregation, combiners significantly reduce the volume of data transfer, leading to faster and more efficient processing. Together, partitioning and combining help to enhance the performance and scalability of the MapReduce framework by optimizing data distribution and minimizing network congestion.

Example:



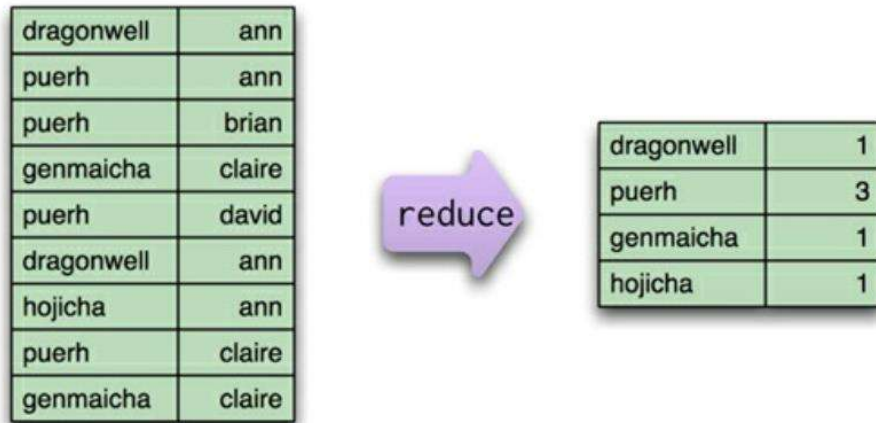
Partitioning allows reduce functions to run in parallel on different keys



Combining reduces data before sending it across the network

A combiner function is, in essence, a reducer function—indeed, in many cases the same function can be used for combining as the final reduction. The reduce function needs a special shape for this to work: Its output must match its input. We call such a function a combinable reducer.

Not all reduce functions are combinable. Eg:

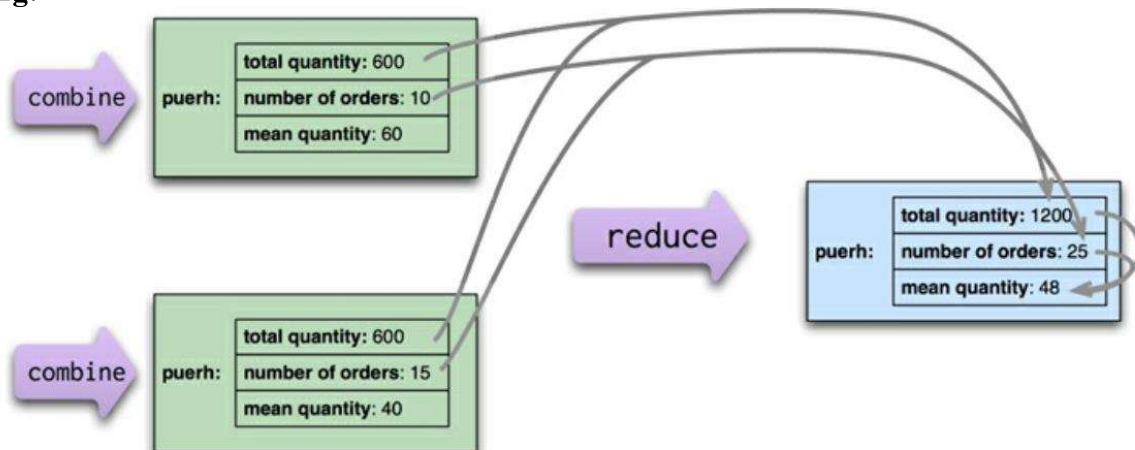


This reduce function, which counts how many unique customers order a particular tea, is not combinable

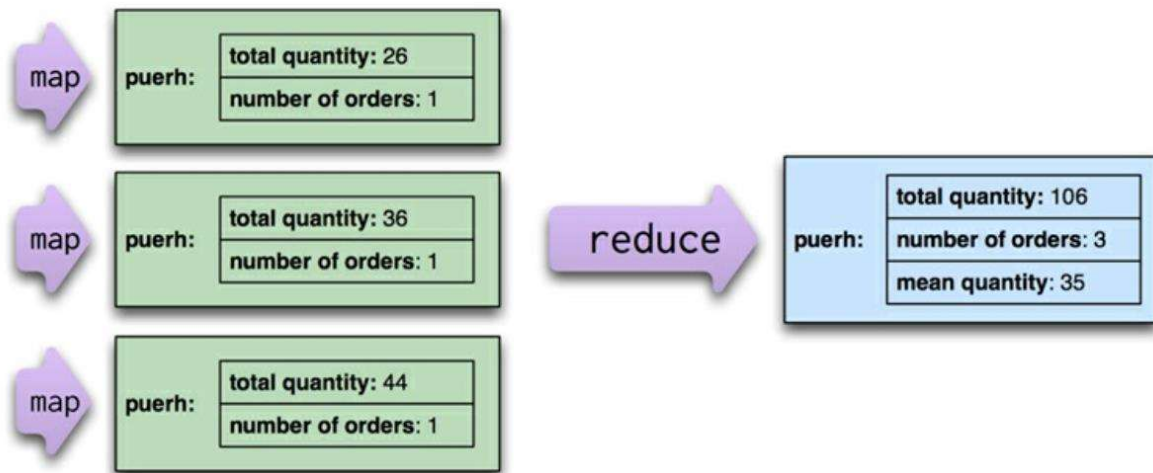
Composing Map-Reduce Calculations

Composing MapReduce calculations involves chaining multiple MapReduce jobs together to perform complex data processing tasks that cannot be accomplished with a single MapReduce job. Each job in the chain takes the output of the previous job as its input, allowing for a series of transformations and aggregations on the data. This composition is essential for tasks like multi-step data processing pipelines, where intermediate results from one phase need further processing in subsequent phases. For example, in a web log analysis scenario, the first MapReduce job might filter and clean the log data, the second job could aggregate the data by user sessions, and the third job might compute statistics on the session data. Effective composition requires careful planning of the intermediate data format and efficient handling of data transfer between jobs. By breaking down complex calculations into a sequence of simpler MapReduce jobs, developers can leverage the power of distributed computing to handle large-scale data processing in a modular and manageable way. This approach also improves fault tolerance, as each stage can be independently retried and debugged.

Eg:



When calculating averages, the sum and count can be combined in the reduce calculation, but the average must be calculated from the combined sum and count.

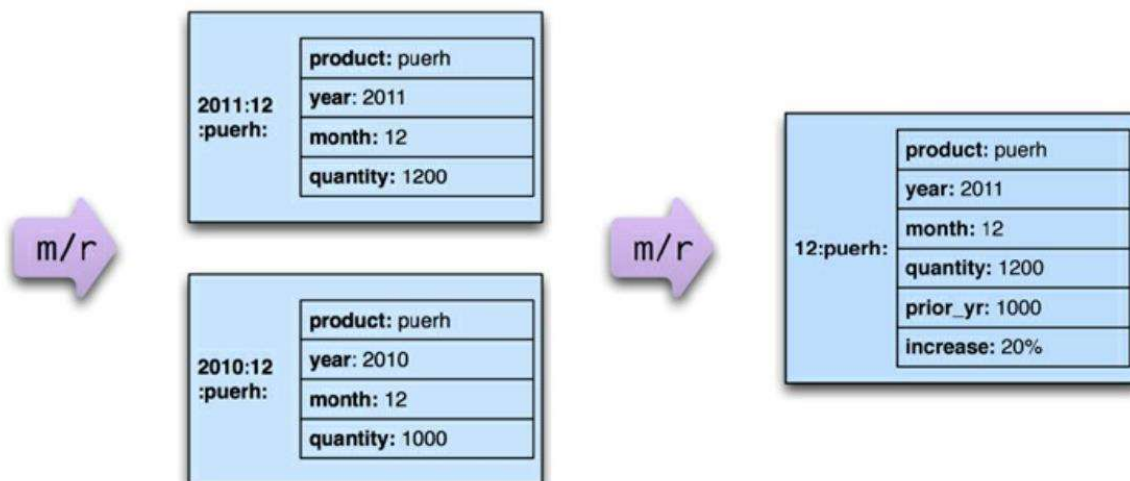


When making a count, each map emits 1, which can be summed to get a total

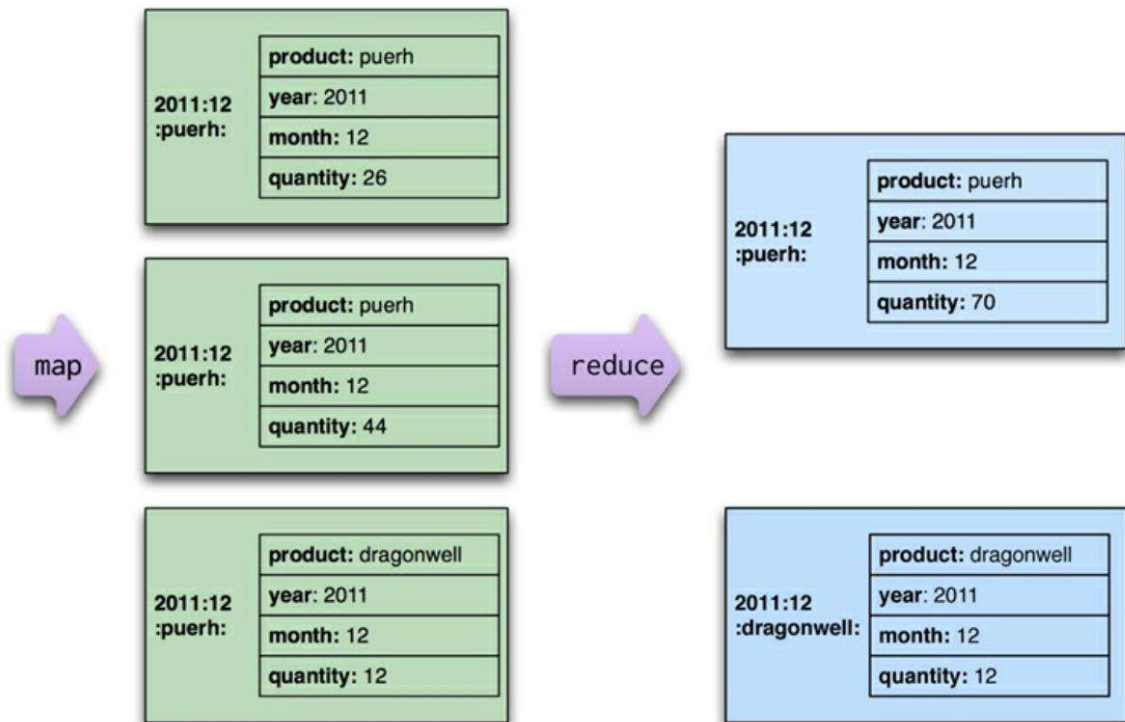
A Two Stage Map-Reduce Example

A two-stage MapReduce process involves executing two consecutive MapReduce jobs to handle complex data processing tasks. In the first stage, the map function processes the raw input data to generate intermediate key/value pairs, and the reduce function aggregates these pairs. For instance, in a word count scenario, the first stage's map function emits each word with a count of one, while the reduce function sums the counts for each word. The output of this stage, consisting of the total counts for each word, serves as the input for the second stage. In the second stage, another map function can further process these aggregated results, such as grouping words by their initial letter or filtering out infrequent words. The reduce function in this stage then performs the final aggregation or transformation, producing the final output. This two-stage approach allows for breaking down complex calculations into manageable steps, enhancing scalability and fault tolerance in distributed computing environments.

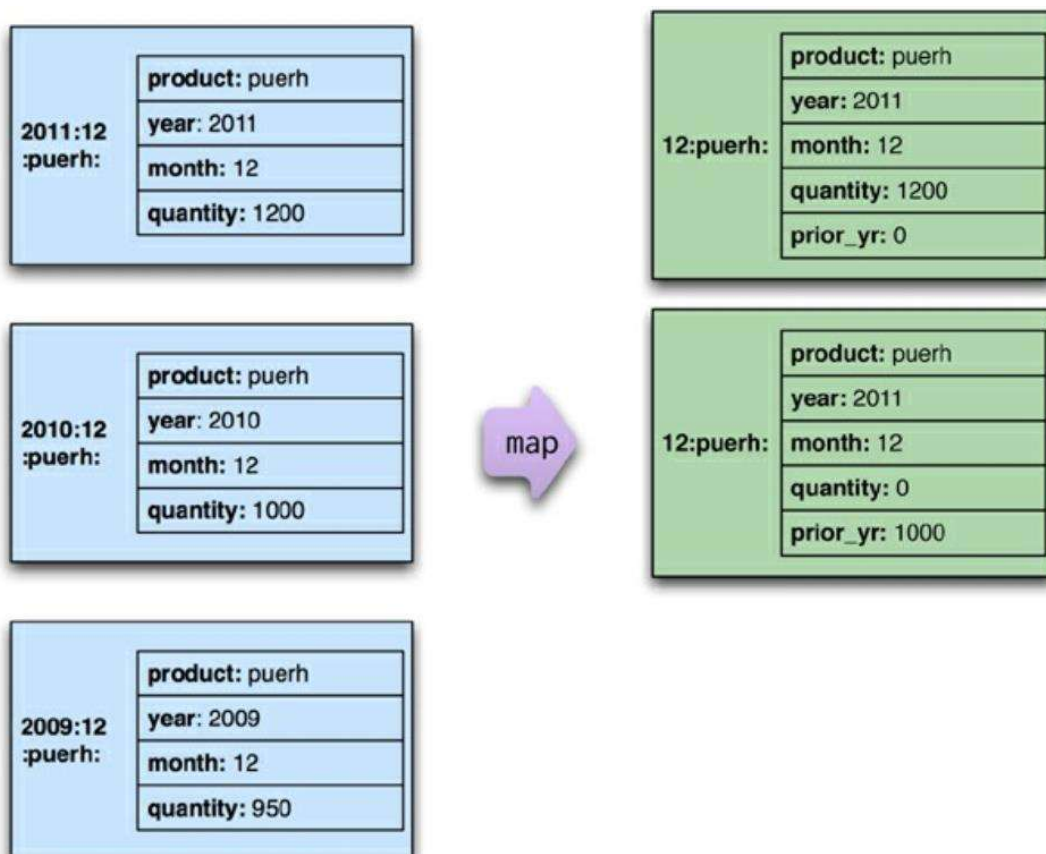
Eg:



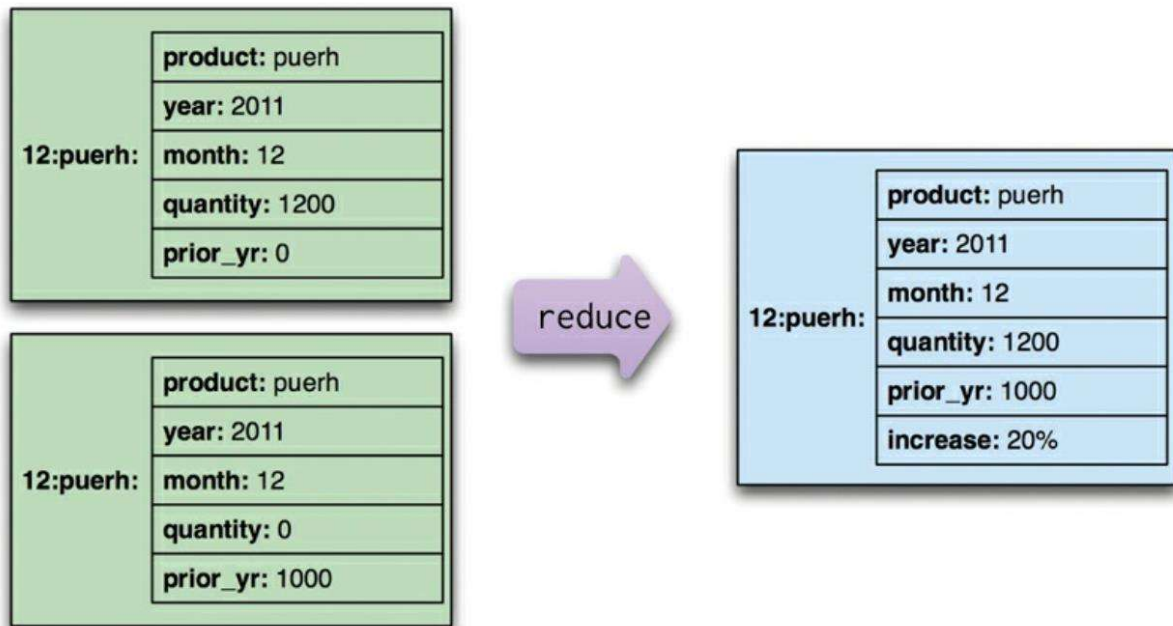
A calculation broken down into two map-reduce steps, which will be expanded in the next three figures



Creating records for monthly sales of a product



The second stage mapper creates base records for year-on-year comparisons.



The reduction step is a merge of incomplete records

Incremental Map-Reduce

- Incremental MapReduce is an extension of the traditional MapReduce framework designed to handle dynamic data by processing only the changes in the dataset rather than reprocessing the entire dataset.
- This approach is particularly useful for applications where data is continuously updated, such as in real-time analytics or iterative machine learning algorithms.
- In incremental MapReduce, the framework identifies and processes only the newly added or modified data since the last computation.
- The map function processes these changes to generate updated intermediate key/value pairs, while the reduce function merges these updates with the previously computed results.
- By focusing on the incremental changes, this method significantly reduces computational overhead and improves efficiency, allowing for faster and more responsive data processing.
- This approach also enhances resource utilization and minimizes latency, making it ideal for applications requiring frequent updates and real-time insights.
- The map stages of a map-reduce are easy to handle incrementally—only if the input data changes does the mapper need to be rerun. Since maps are isolated from each other, incremental updates are straightforward.

Key-Value Databases

Key-value databases are a type of NoSQL database that store data as a collection of key-value pairs, where each key is unique and directly associated with a specific value. This simple and flexible data model allows for high performance and scalability, making key-value databases ideal for applications that require fast read and write operations, such as caching, session management, and real-time analytics. The value in a key-value database can be any type of data, ranging from simple strings to complex objects like JSON or binary data. Operations are performed using keys, which enables quick retrieval of values without the need for complex query language or table scans. Examples of key-value databases